

Projeto e Análise de Algoritmos

Fluxo Máximo em Grafos

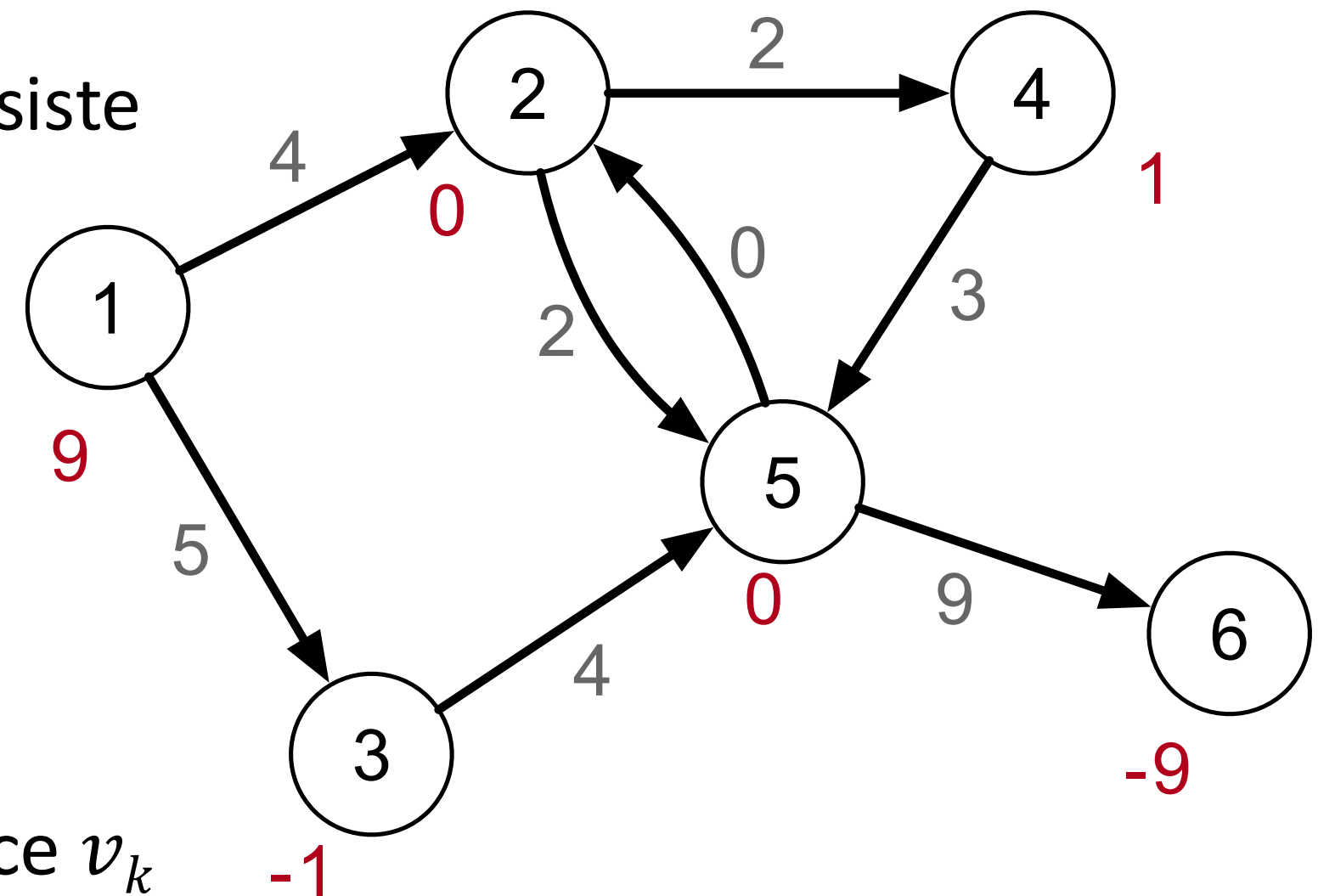
Thiago Pinheiro de Araújo

EMAp/FGV

2025.2

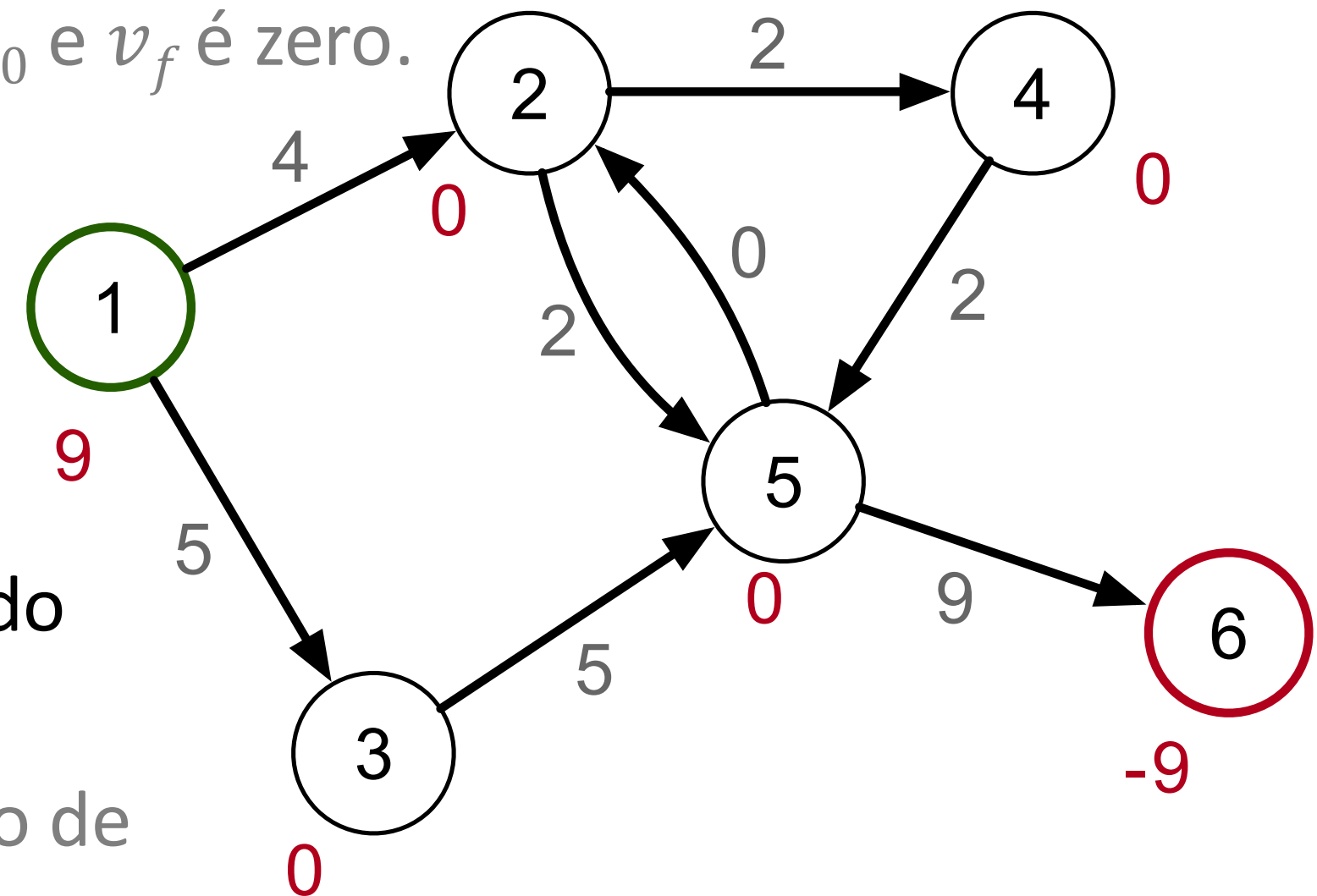
Apresentar o conceito de **fluxo em grafos**, o problema do fluxo máximo, o método de Ford-Fulkerson e um exemplo de implementação.

- O **fluxo de uma aresta** e_{ij} de um grafo $G = (V, E)$ consiste em um valor inteiro positivo f_{ij} associado à ela.
- O fluxo de entrada (*inflow*) de um vértice consiste no somatório do fluxo das suas arestas de entrada
 - Analogamente, o fluxo de saída (*outflow*) de um vértice consiste no somatório do fluxo das suas arestas de saída.
- O saldo de um fluxo (*net flow*) f_k de um vértice v_k é a diferença entre o fluxo de saída e o fluxo de entrada.

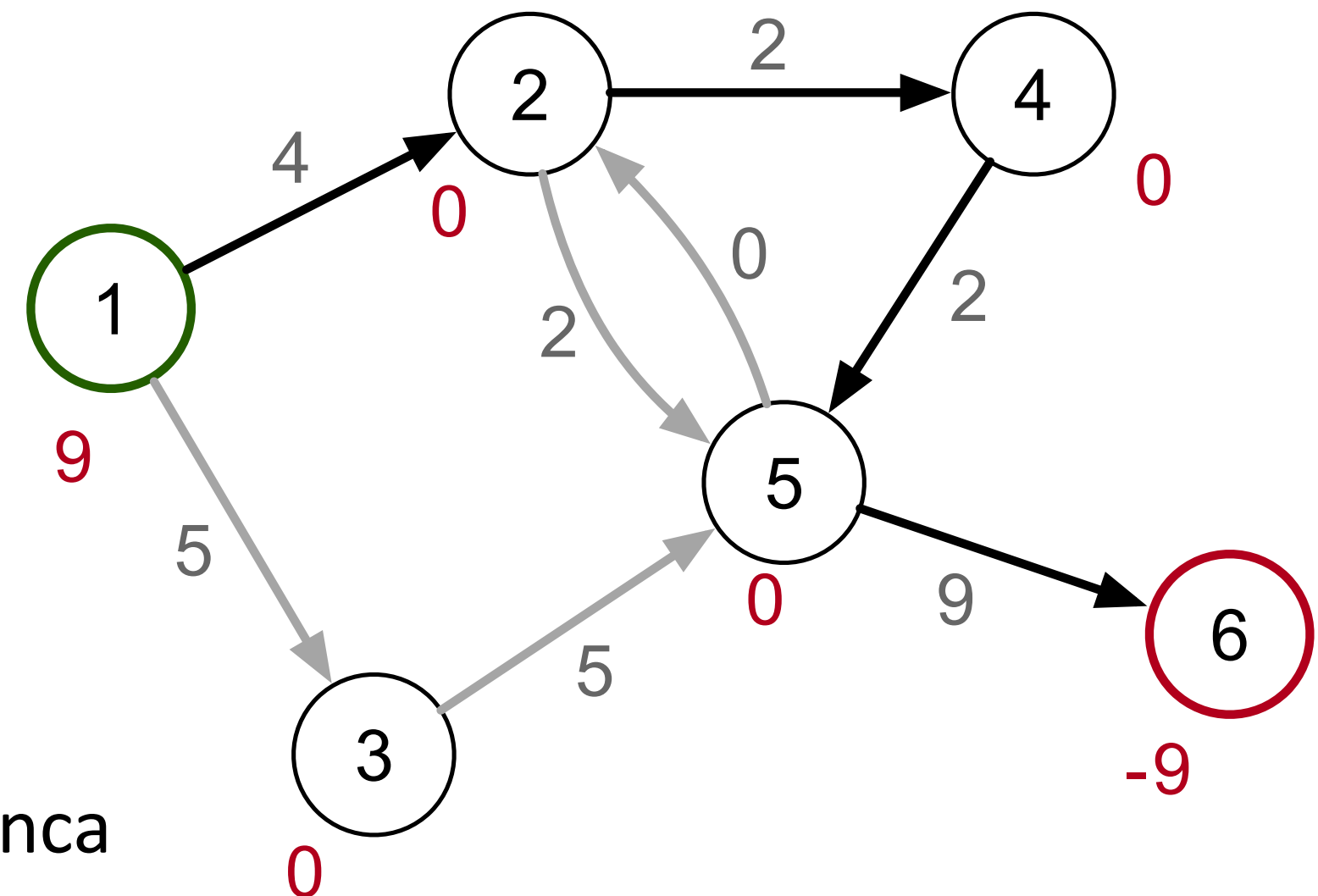


Introdução

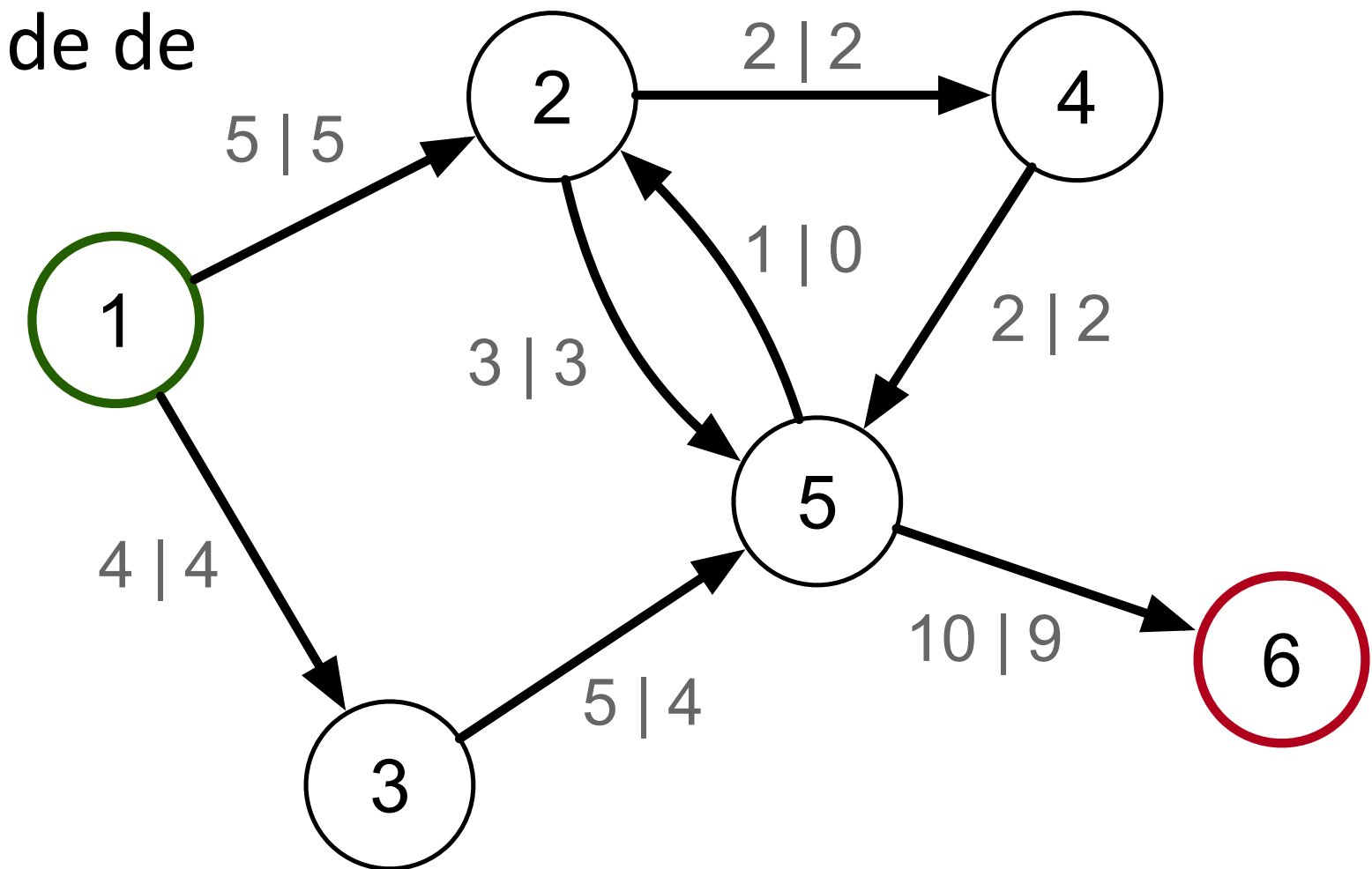
- Considerando um vértice inicial v_0 e um vértice final v_f , o **fluxo** (*flow*) de um grafo $G = (V, E)$ consiste na definição de um fluxo f_{ij} para cada aresta e_{ij} tal que:
 - O saldo em v_0 é positivo.
 - O saldo em qualquer vértice diferente de v_0 e v_f é zero.
- Se v_0 for uma fonte e v_f for um sorvedouro o saldo de v_f será $-f_0$.
- A intensidade de um fluxo é definida pelo saldo no vértice inicial f_0
 - Se o fluxo de entrada em v_0 for zero, o fluxo de saída terá o valor da intensidade.



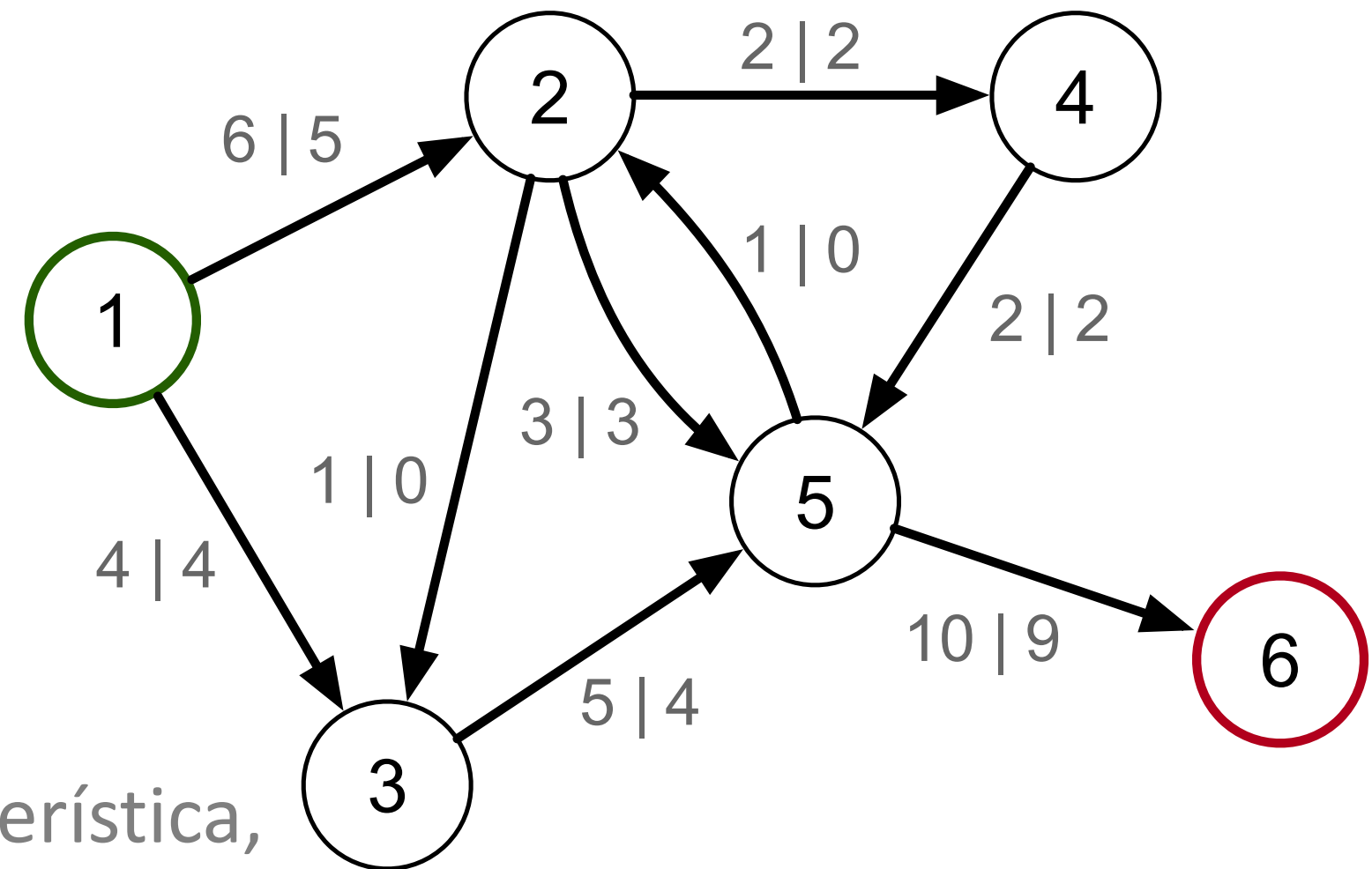
- Um fluxo pode ser desmembrado por uma coleção de caminhos que conduzem uma quantidade de fluxo u_k
 - O caminho destacado na figura ao lado conduz 2 unidades.
- Todo caminho é simples, inicia em v_0 e termina em v_f .
- A soma das quantidades de fluxo de todos os caminhos é igual à intensidade do fluxo.
- A soma das unidades u_{ij} de uma aresta e_{ij} nunca será maior que o fluxo f_{ij} .



- Uma **rede** (*flow network*) é um grafo $G = (V, E)$ em que cada aresta possui uma capacidade c_{ij} representada por um número positivo.
- A capacidade de uma aresta limita a quantidade de fluxo que pode passar nela.
 - Logo, um fluxo f_{ij} respeita a capacidade c_{ij} caso $f_{ij} \leq c_{ij}$.
- Uma aresta está saturada caso $f_{ij} = c_{ij}$.
 - Uma aresta está vazia caso $f_{ij} = 0$.



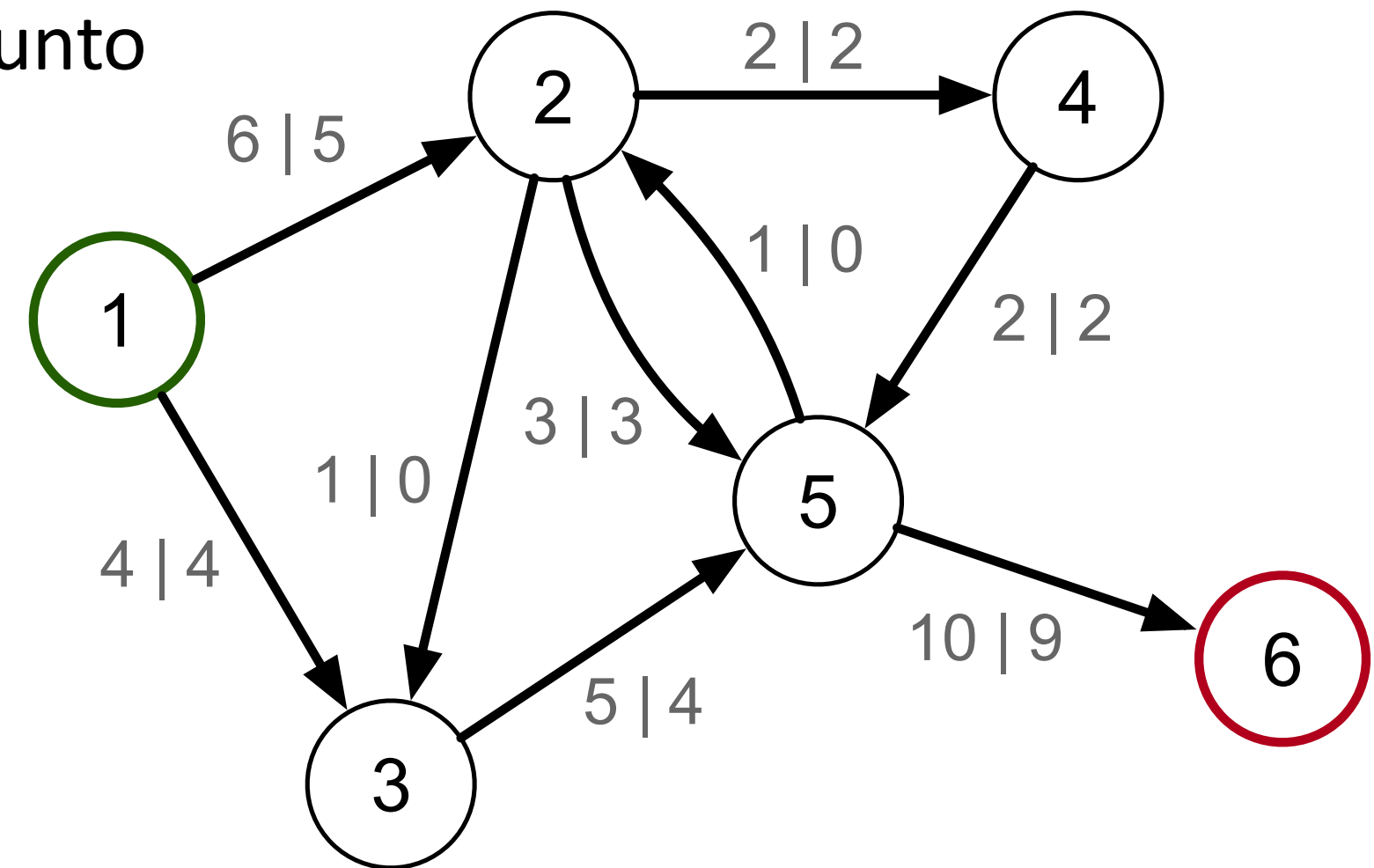
- **Problema:** dado um grafo $G = (V, E)$ representando uma rede com capacidade nas arestas e vértices de origem e destino, encontre o fluxo máximo respeitando a capacidade das arestas.
- O fluxo será máximo caso a intensidade do mesmo seja a maior possível no grafo G .
- Se todo caminho (que conduz fluxo) de v_0 até v_f possuir ao menos uma aresta saturada o fluxo poderá ser considerado máximo?
 - Não! Todo fluxo máximo possui essa característica, no entanto a recíproca não é verdadeira.



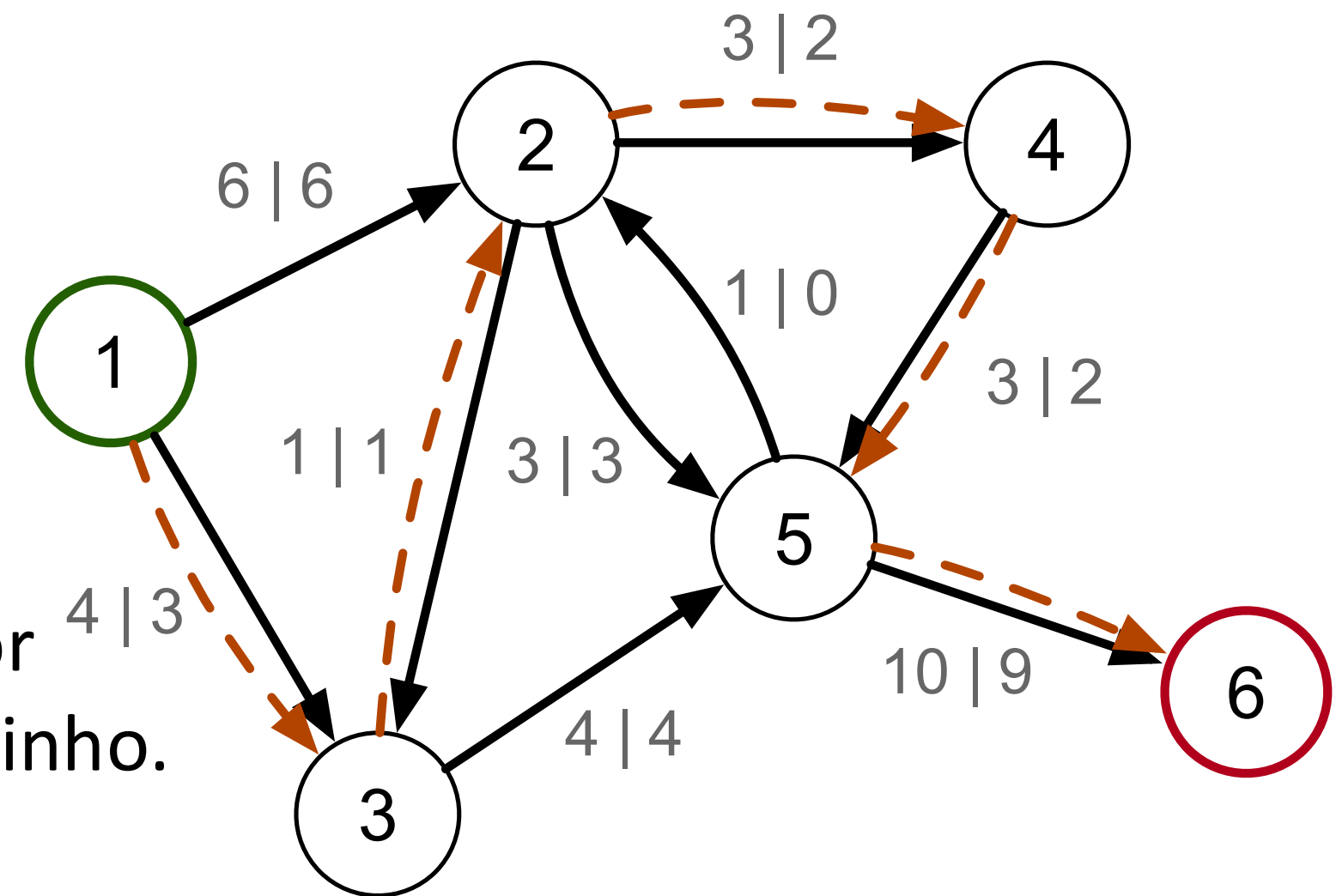
Método de Ford-Fulkerson

Método de Ford-Fulkerson

- O **método de Ford-Fulkerson** foi publicado por Lester Randolph Ford Jr. e Delbert Ray Fulkerson em 1956.
- Esse método define uma estrutura e um conjunto de conceitos com o objetivo de solucionar o problema de fluxo máximo em uma rede.
- A estratégia consiste em encontrar caminhos que aumentem a intensidade do fluxo até atingir a capacidade máxima da rede.

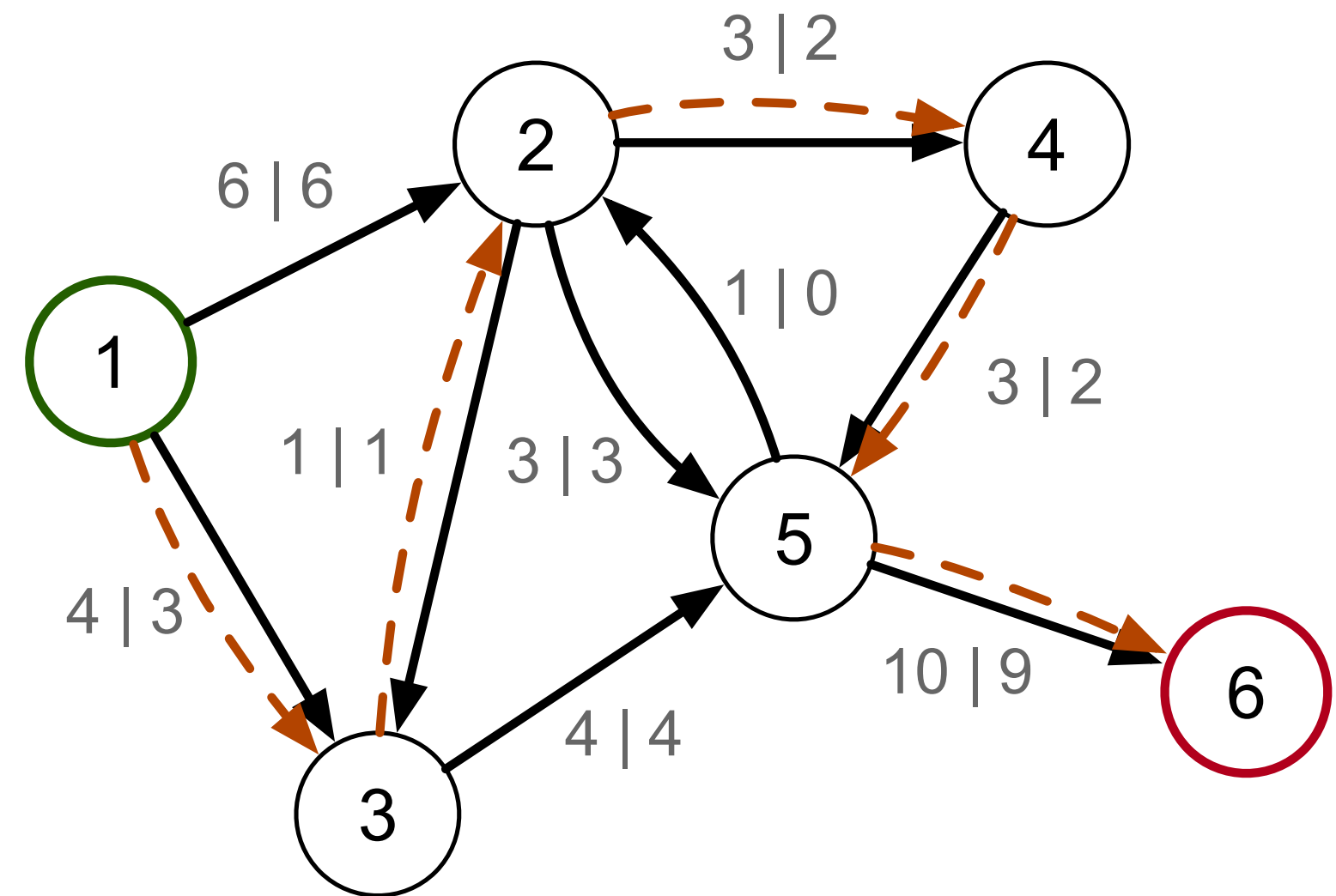


- Um **pseudocaminho** em um grafo $G = (V, E)$ consiste em uma sequência de vértices tal que para cada par de vértices consecutivos (v_i, v_j) existe uma aresta e_{ij} ou e_{ji} em G .
- Uma aresta e_{ij} de um pseudocaminho é considerada uma aresta direta se $e_{ij} \in G$.
 - É uma aresta reversa caso contrário.
- Se o pseudocaminho for composto apenas por arestas diretas é considerado apenas um caminho.

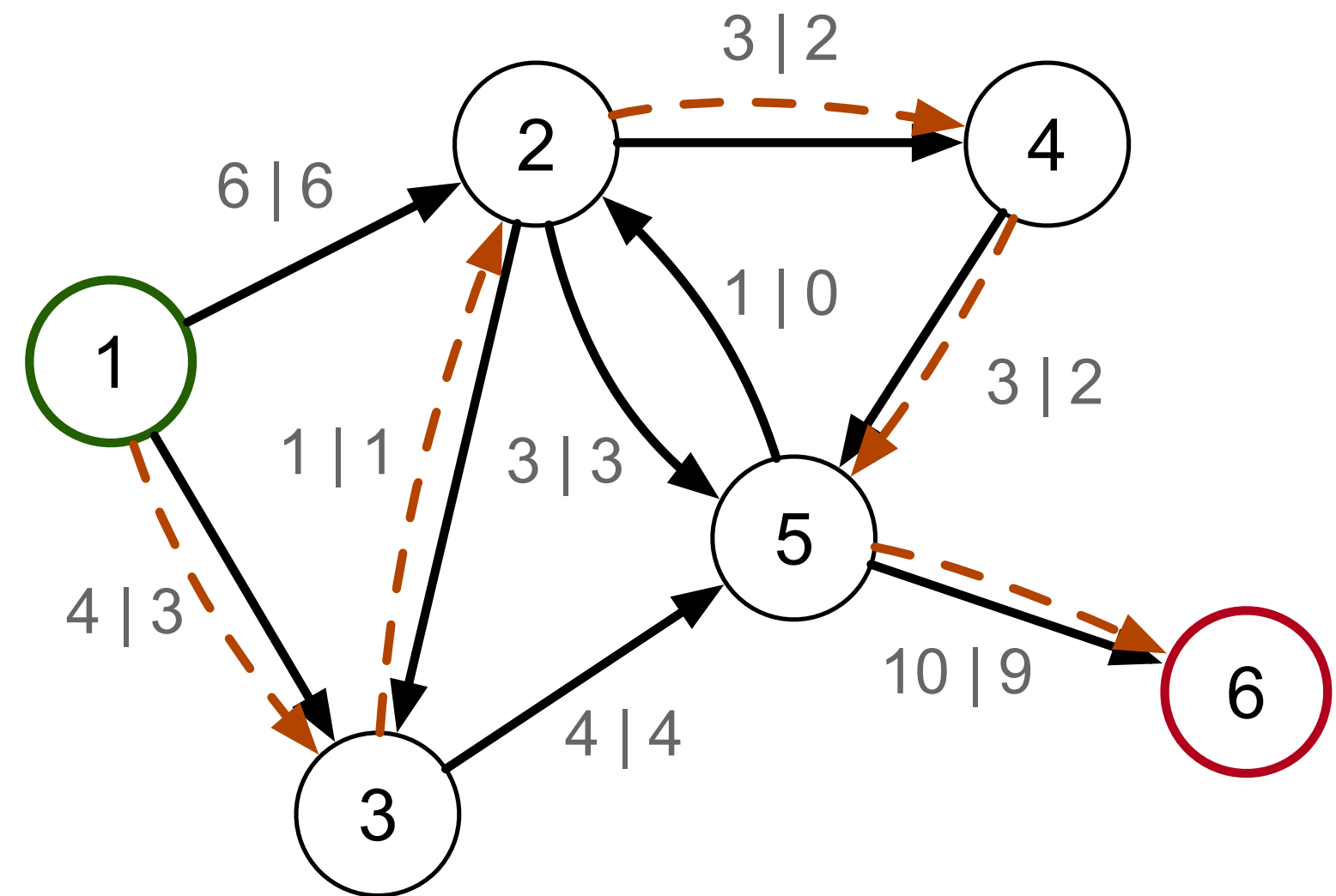


Método de Ford-Fulkerson

- Um **pseudocaminho aumentador** é um pseudocaminho que possui as seguintes características nas arestas que o compõe:
 - Nenhuma aresta direta está saturada.
 - Nenhuma aresta reversa está vazia.
- A adição de um pseudocaminho aumentador à um fluxo f consiste em enviar u unidades de fluxo ao longo do caminho:
 - $f_{ij} = f_{ij} + u$ se for uma aresta direta.
 - $f_{ij} = f_{ij} - u$ se for uma aresta reversa.
- O fluxo resultante possui uma intensidade incrementada de u unidades.



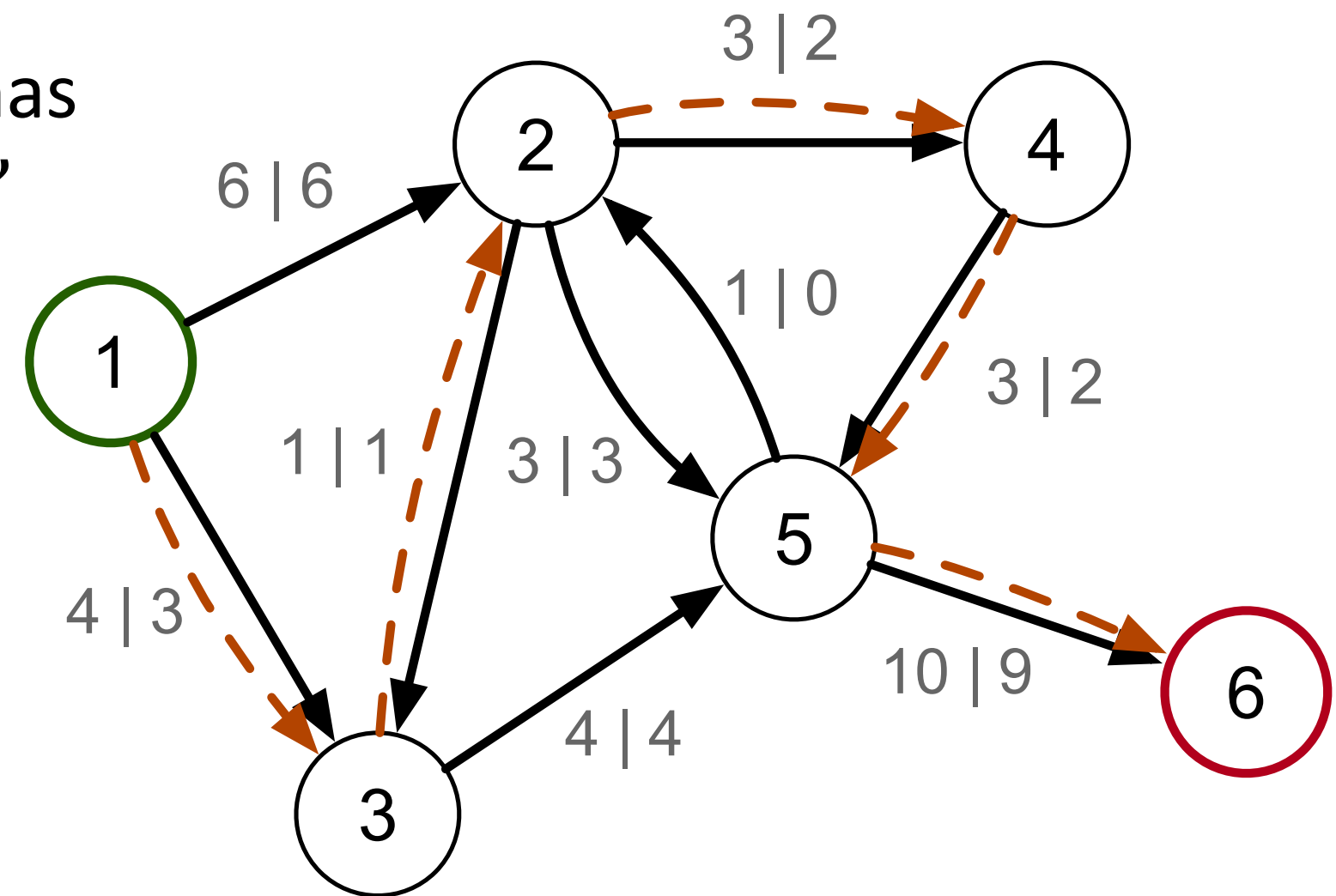
- A **capacidade residual** δ_{ij} de uma aresta e_{ij} em um pseudocaminho é definida pela diferença entre a sua capacidade e o fluxo corrente δ
 - $\delta_{ij} = c_{ij} - f_{ij}$, em uma aresta direta.
 - $\delta_{ij} = f_{ji}$, em uma aresta reversa.
- A capacidade residual de um pseudocaminho é definida pela menor das capacidades residuais das suas arestas.



Fluxo máximo em Grafos

Método de Ford-Fulkerson

- Seja P um pseudocaminho aumentador de f em $G = (V, E)$ com capacidade residual δ .
- Ao adicionar u unidades (sendo $0 < u \leq \delta$) nas arestas do caminho P temos um novo fluxo f' que respeita as capacidades de G .
- A intensidade de f' será u unidades maior que a capacidade original.



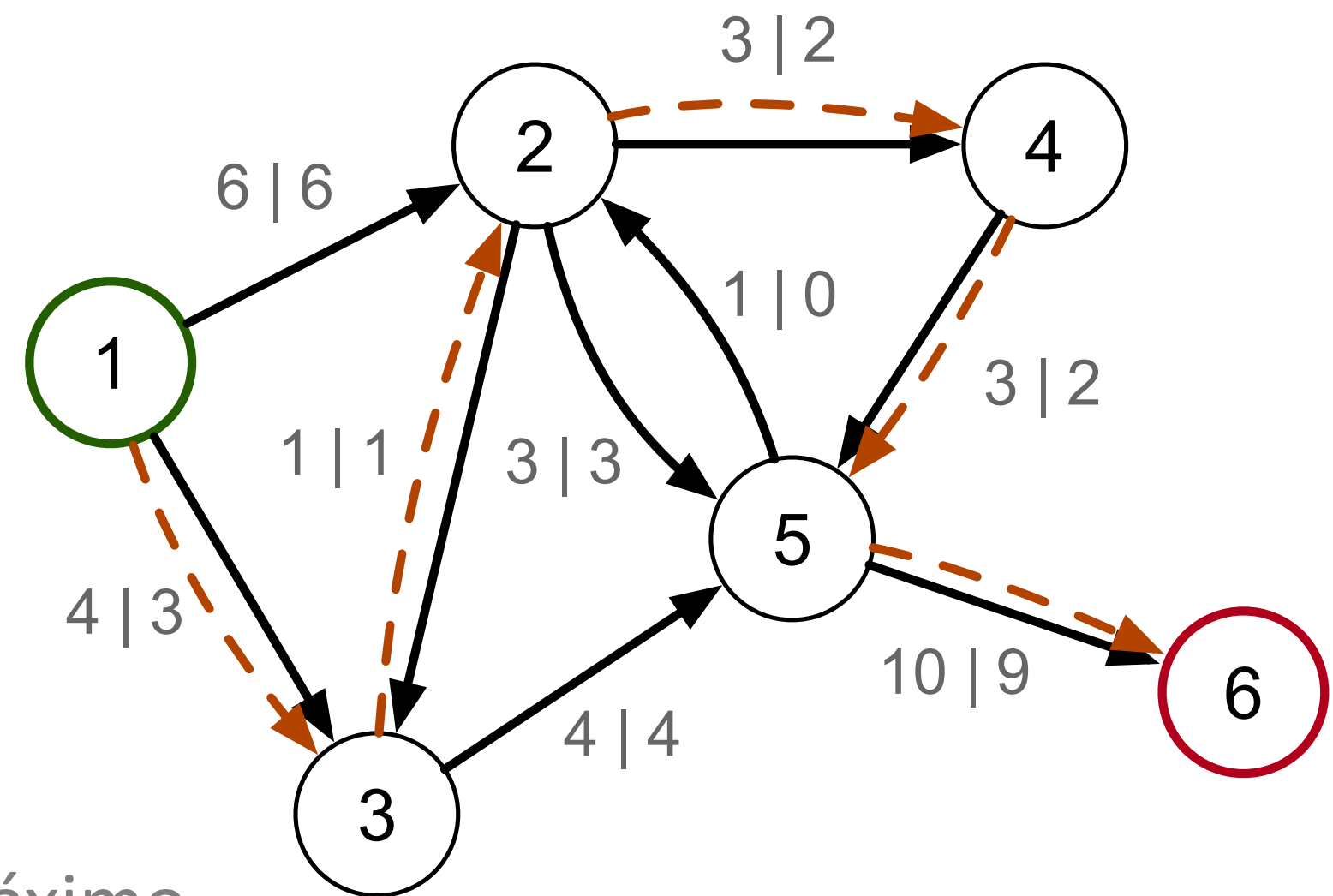
Fluxo máximo em Grafos

Método de Ford-Fulkerson

- O método de Ford-Fulkerson soluciona o problema adicionando à cada iteração um pseudocaminho aumentador até atingir o fluxo máximo.

- Ideia geral:

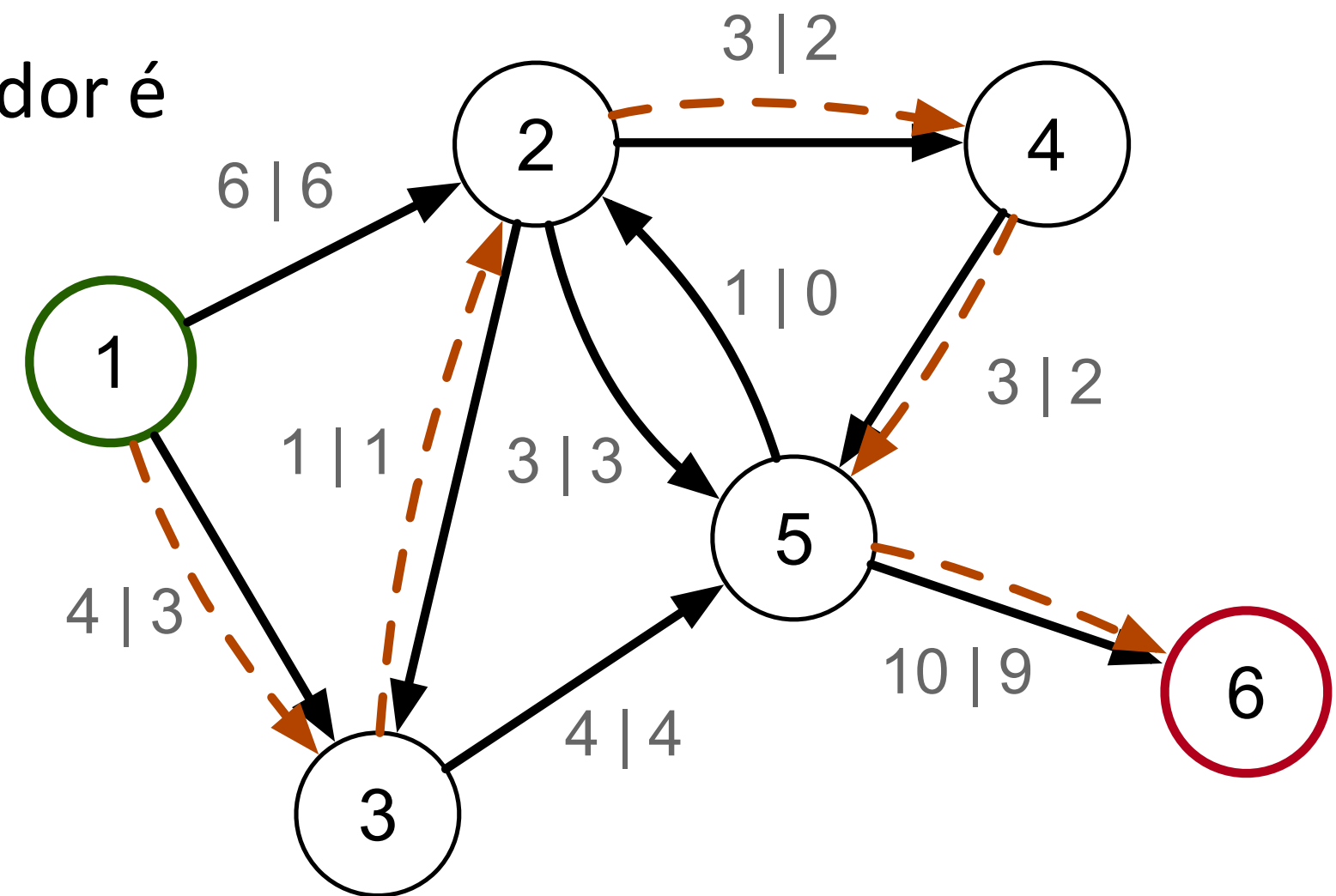
1. Defina um fluxo f nulo.
2. Encontre um pseudocaminho aumentador P .
3. Calcule a capacidade residual δ de P .
4. Envie δ unidades pelo caminho P .
5. Atualize o fluxo f .
6. Repita o passo 2 até encontrar o fluxo máximo.



Fluxo máximo em Grafos

Método de Ford-Fulkerson

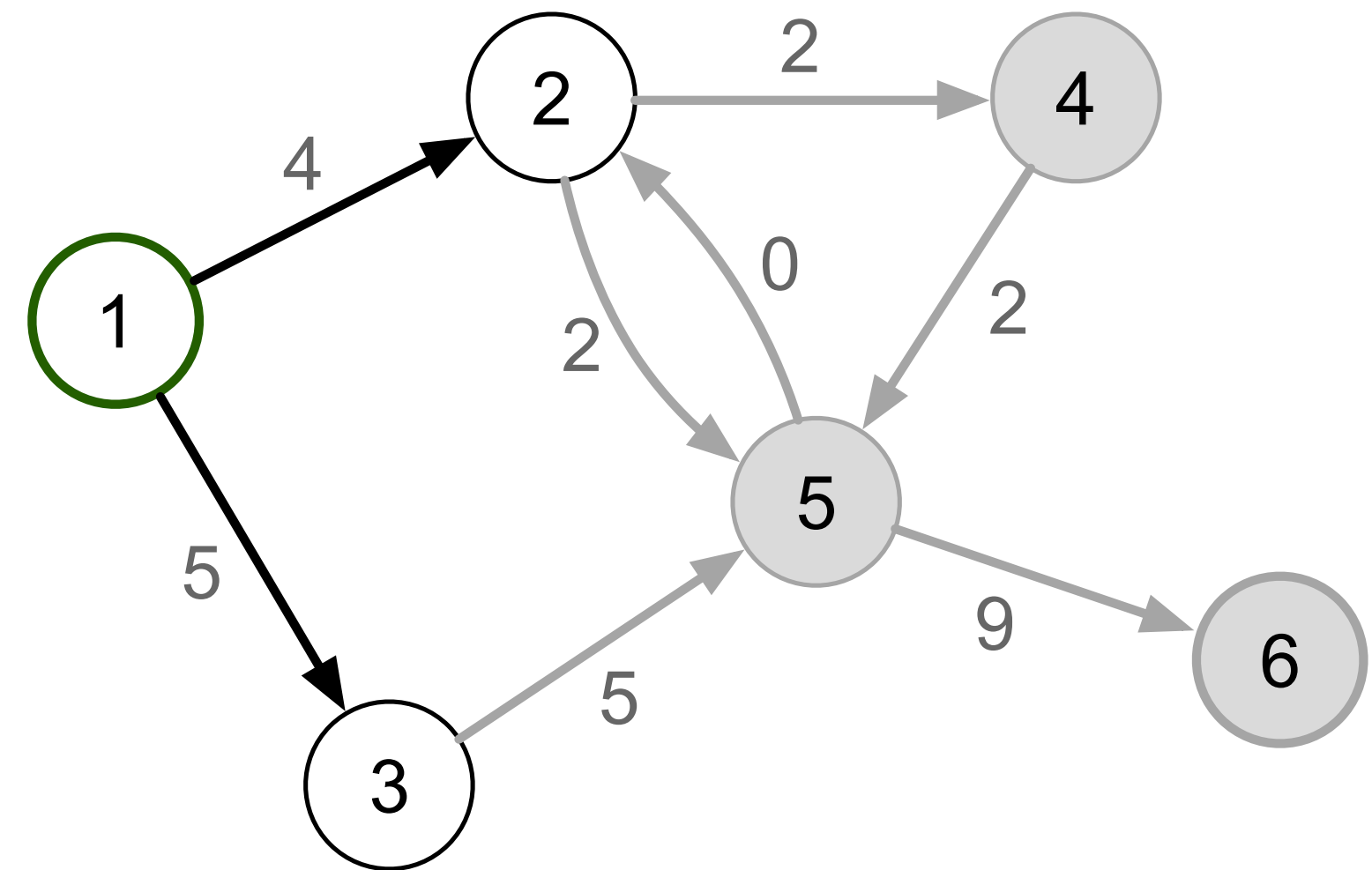
- Mesmo sem definir a estratégia de busca pelo caminho aumentador podemos fazer uma avaliação da sua complexidade.
- A cada iteração um pseudocaminho aumentador é adicionado com pelo menos uma unidade.
 - A pesquisa por esse pseudocaminho custa pelo menos $O(E)$.
- Sendo f_{max} o fluxo máximo, temos uma complexidade $O(Ef_{max})$.



Fluxo máximo em Grafos

Método de Ford-Fulkerson

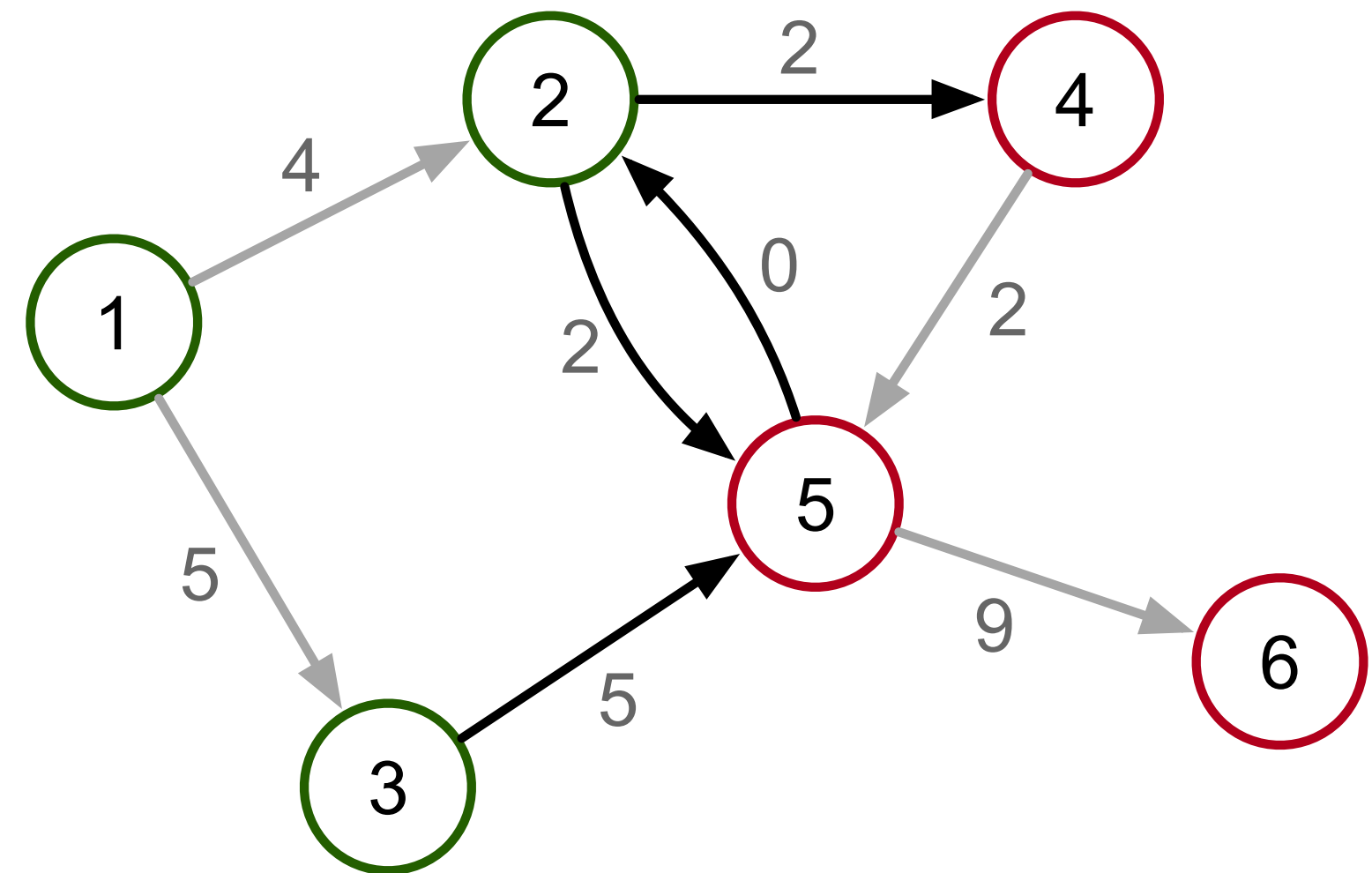
- O conceito de **saldo de fluxo** pode ser estendido à um conjunto de vértices S de G .
- O fluxo de saída é definido pela soma do fluxo de todas as arestas que saem de S .
- O fluxo de entrada é definido pela soma do fluxo de todas as arestas que entram em S .
- O saldo é a diferença entre o fluxo de saída e o fluxo de entrada de S .



Fluxo máximo em Grafos

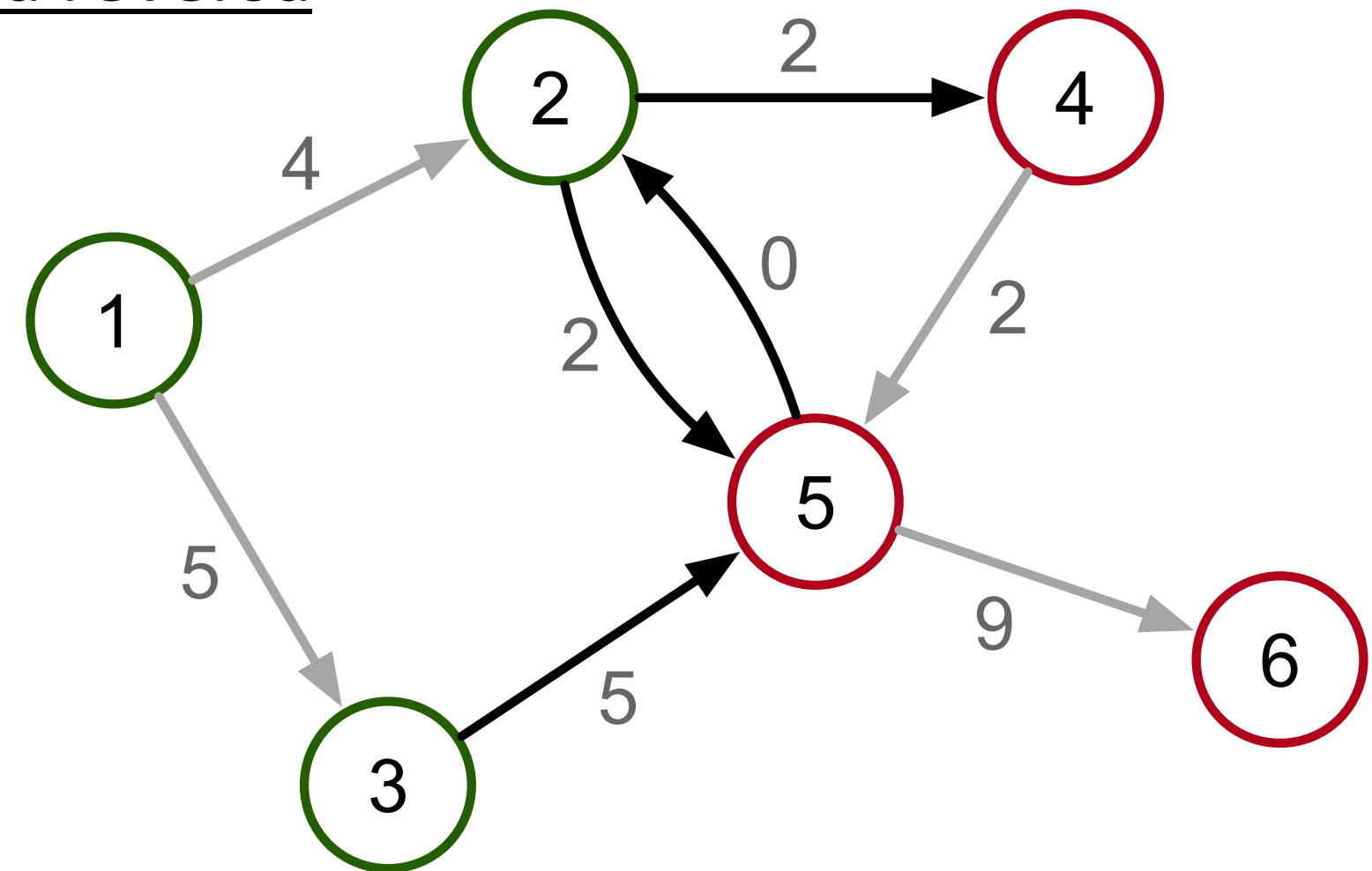
Método de Ford-Fulkerson

- Considere um **corte** C um conjunto de arestas que separa o grafo em duas componentes V_1 e V_2 de forma que:
 - V_1 contém o vértice v_0 .
 - V_2 contém o vértice v_f .
- V_1 é a margem superior do corte
- V_2 é a margem inferior do corte.



Método de Ford-Fulkerson

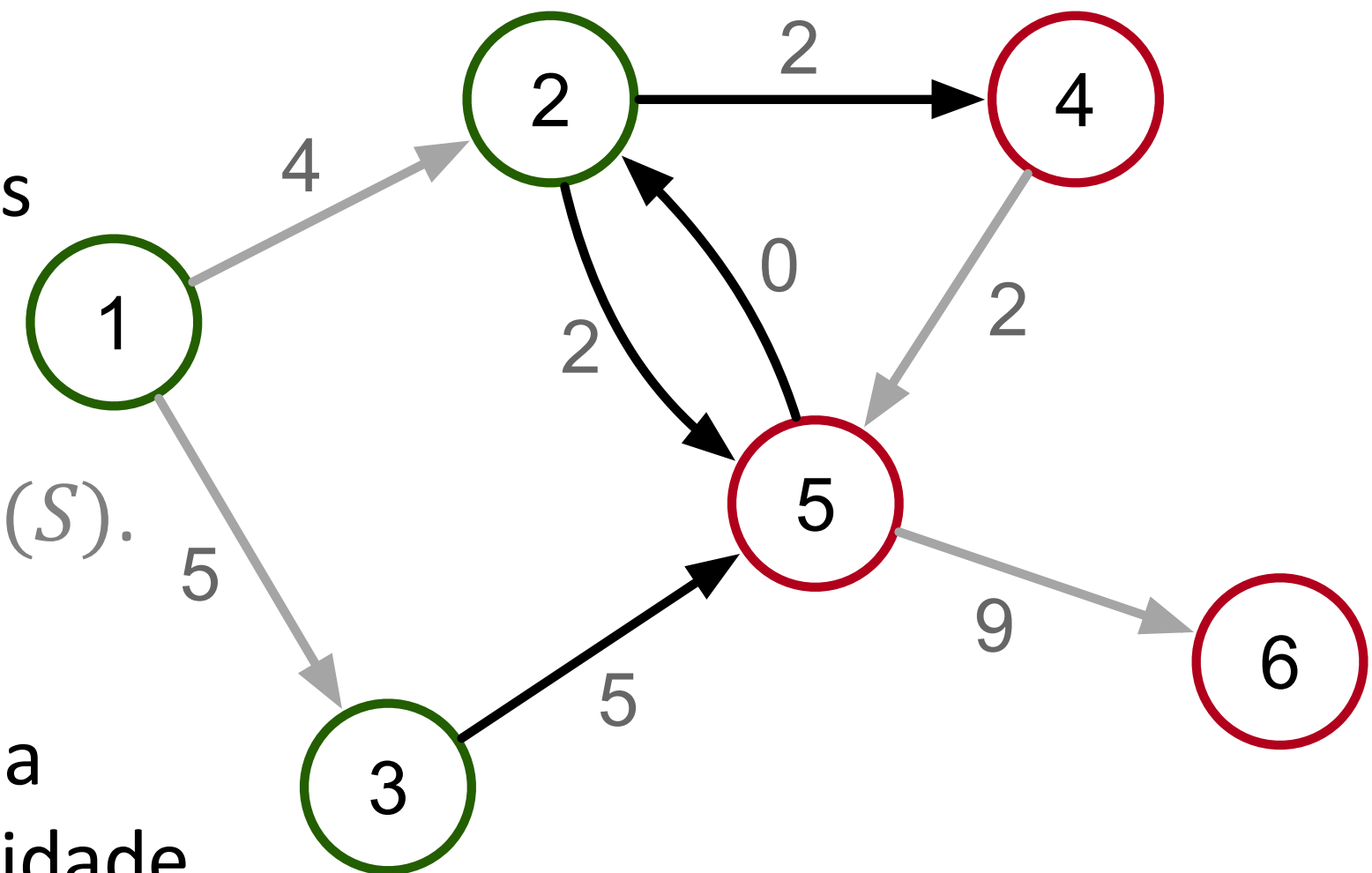
- A aresta $e = (v_i, v_j)$ de um corte é uma aresta direta se $v_i \in V_1$ e $v_j \in V_2$.
- A aresta $e = (v_i, v_j)$ de um corte é uma aresta reversa se $v_i \in V_2$ e $v_j \in V_1$.
- A **capacidade de um corte** é definida pela soma das capacidades das arestas diretas
 - Arestas reversas não influenciam a capacidade do corte.
- O corte mínimo de uma rede é o corte C_k cuja capacidade é mínima dentre todos os cortes.



- A capacidade de um corte C limita a intensidade de um fluxo f que respeita as capacidades
 - $intensidade(f) \leq capacidade(C)$.

- A intensidade do fluxo no conjunto de vértices da margem superior S de um corte é igual ao saldo de f nesse conjunto de vértices
 - $intensidade(f) = outflow(S) - inflow(S)$.

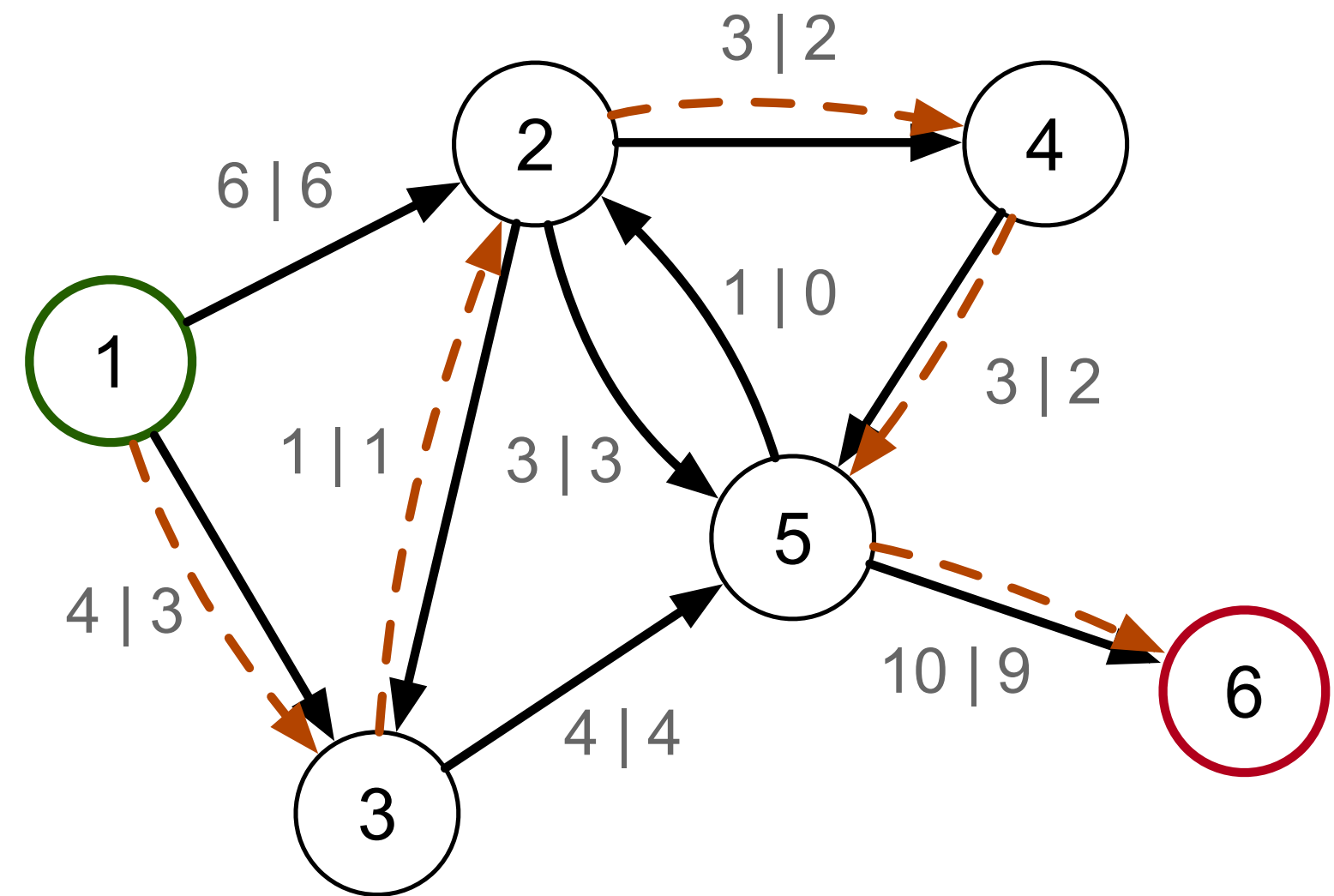
- Portanto, observe que a intensidade é limitada pelo fluxo de saída de S , que respeita a capacidade do corte C .



Fluxo máximo em Grafos

Método de Ford-Fulkerson

- A intensidade de qualquer fluxo é igual ou menor à capacidade de qualquer corte.
- Se tivermos um fluxo com intensidade igual à capacidade de um corte esse fluxo será máximo para esse corte
 - Podemos utilizar essa propriedade para identificar quando o algoritmo encontrou o fluxo máximo.
- Portanto, a intensidade de um fluxo máximo é igual à capacidade do corte mínimo.



Implementação do algoritmo

Implementação do algoritmo

- A implementação do algoritmo de Ford-Fulkerson depende da definição de um algoritmo para encontrar o próximo pseudocaminho aumentador, e algumas evoluções na estrutura de dados
 - A seguir iremos evoluir a representação baseada em listas de adjacências.
- Considere os seguintes atributos no nó da lista (aresta):
 - Referência para o outro vértice
 - Próximo nó da lista
 - Capacidade da aresta *[novo]*
 - Fluxo na aresta *[novo]*

Implementação do algoritmo

- Como o algoritmo depende de **pseudocaminhos**, precisamos ser capazes de representá-los através da estrutura de dados.
- Para tanto criaremos para cada aresta e_{ij} uma aresta virtual e_{ji} .
- Toda aresta virtual será marcada como tal através de um novo atributo na aresta representando seu tipo.
- Toda aresta possuirá uma referência para sua aresta virtual (e vice-versa).

Implementação do algoritmo

- A seguir um exemplo de implementação da estrutura de dados:

```
enum EdgeType {  
    Original,  
    Virtual  
};
```

```
class EdgeNode {  
public:  
    ...  
    EdgeType type() { return m_type; }  
  
    EdgeNode * reference() { return m_reference; }  
    void setReference(EdgeNode * value) { m_reference = value; }  
  
    int capacity() { return m_capacity; }  
    void setCapacity(int value) { m_capacity = value; }  
  
    int flow() { return m_flow; }  
    void setFlow(int value) { m_flow = value; }  
    void addFlow(int value) { m_flow += value; }  
    void removeFlow(int value) { m_flow -= value; }  
  
private:  
    int m_otherVertex;  
    EdgeNode * m_next;  
    int m_cost;  
    EdgeType m_type;  
    EdgeNode * m_reference;  
    int m_capacity;  
    int m_flow;  
};
```

Implementação do algoritmo

- Ideia geral do algoritmo:
 1. Configure o grafo
 - Inicializando as propriedades de cada nó.
 - Adicionando as arestas virtuais.
 2. Calcule o fluxo máximo
 - Defina um fluxo f nulo.
 - Encontre um pseudocaminho aumentador P .
 - Calcule a capacidade residual δ de P .
 - Envie δ unidades pelo caminho P
 - Atualize o fluxo f .
 - Repita até atingir o fluxo máximo.
 3. Remova as arestas virtuais do grafo.

Implementação do algoritmo

■ Exemplo de implementação:

```
int computeMaxFlow(vertex v0, vertex vf) {
    vertex parent[m_numVertices];
    EdgeNode * parentEdge[m_numVertices];
    maxFlowSetup();
    int maxFlow = 0;
    while (true) {
        int delta = findNextAugmentingPath(
            v0, vf, parent, parentEdge);
        if (!delta) break;
        for (vertex v2=vf; v2 != v0; v2 = parent[v2]) {
            EdgeNode * edge = parentEdge[v2];
            edge->addFlow(delta);
            edge->reference()->removeFlow(delta);
        }
        maxFlow += delta;
    }
    maxFlowTearDown();
    return maxFlow;
}
```

```
void maxFlowSetup() {
    for (vertex v=0; v < m_numVertices; v++) {
        EdgeNode * edge = m_edges[v];
        while (edge) {
            if (edge->type() == Original) {
                EdgeNode * virtualEdge = new EdgeNode(
                    v, Virtual, edge);
                edge->setReference(virtualEdge);
                virtualEdge->setNext(
                    m_edges[edge->otherVertex()]);
                m_edges[edge->otherVertex()] = virtualEdge;
            }
            edge = edge->next();
        }
    }
}

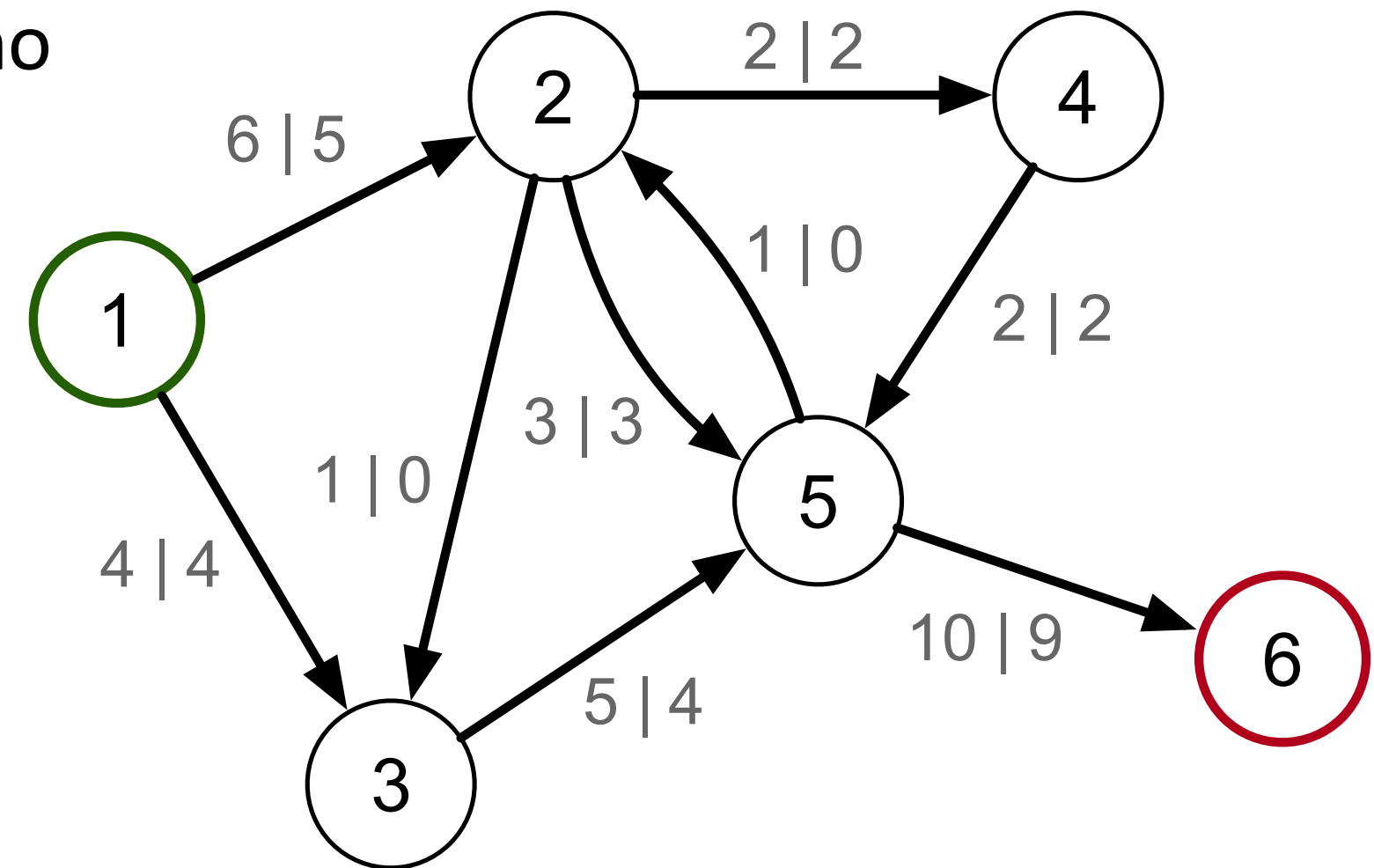
void maxFlowTearDown() {
    for (vertex v=0; v < m_numVertices; v++) {
        EdgeNode * edge = m_edges[v];
        while (edge) {
            if (edge->type() == Virtual) {
                EdgeNode * temp = edge;
                m_edges[v] = edge->next();
                delete temp;
            }
            edge = edge->next();
        }
    }
}
```

Implementação Edmonds-Karp

Fluxo máximo em Grafos

Algoritmo de Edmond-Karp

- O **algoritmo de Edmond-Karp** (1972) fornece um algoritmo para encontrar pseudocaminhos aumentadores, a ser aplicado no método de Ford-Fulkerson.
- Esse algoritmo é capaz de encontrar o próximo pseudocaminho aumentador com tamanho mínimo sobre um grafo $G = (V, E)$ com capacidade nas arestas.
- A estratégia desse algoritmo é baseada no algoritmo de busca em largura (BFS).



- Ideia geral do algoritmo:
 - Adiciona o vértice v_0 em uma fila.
 - Enquanto a fila não for vazia
 - Remove o vértice v_i da fila.
 - Se $v_i = v_f$, sai do loop.
 - Para cada vértice adjacente v_j .
 - Se ainda não foi visitado e a capacidade residual for maior que zero
 - Define o vértice pai e adiciona v_j na fila.
 - Se v_f não foi visitado
 - Retorna zero.
 - Percorre o caminho mínimo usando o vetor de pais calculando o menor δ .
 - Retorna δ .

- Exemplo de implementação:

```
int findNextAugmentingPath(vertex v0, vertex vf, vertex * parent,
                           EdgeNode * parentEdge[]) {
    bool visited[m_numVertices];
    for (vertex v=0; v < m_numVertices; v++) { visited[v] = false; }
    queue<int> queue;
    visited[v0] = true; queue.push(v0);
    while (!queue.empty()) {
        int v = queue.front(); queue.pop();
        if (v == vf) break;
        EdgeNode * edge = m_edges[v];
        while(edge) {
            int residual = edge->capacity() - edge->flow();
            int v2 = edge->otherVertex();
            if (residual > 0 && !visited[v2]) {
                visited[v2] = true; parent[v2] = v; parentEdge[v2] = edge;
                queue.push(v2);
            }
            edge = edge->next();
        }
    }
    if (!visited[vf]) return 0;
    int delta = INT_MAX;
    for (vertex v2 = vf; v2 != v0; v2 = parent[v2]) {
        EdgeNode * edge = parentEdge[v2];
        delta = min(delta, edge->capacity() - edge->flow());
    }
    return delta;
}
```

- Consideramos uma aresta **crítica** em um pseudocaminho P aumentador se a sua capacidade residual for igual à do pseudocaminho
 - Ao adicionar o pseudocaminho ao fluxo a capacidade residual de toda aresta crítica no caminho será zero.
- Todo pseudocaminho aumentador possui um tamanho maior ou igual ao pseudocaminho anterior.
- Se uma aresta for crítica em um pseudocaminho aumentador com tamanho k , ela somente poderá ser crítica em um pseudocaminho aumentador com tamanho maior que $k + 2$.

- Todo caminho aumentador tem tamanho menor que V .
- Logo uma aresta pode ser crítica no máximo $\frac{V}{2}$ vezes.
- Cada pseudocaminho irá definir ao menos uma aresta crítica, logo temos um total de $E \frac{V}{2}$ pseudocaminhos.
- Considerando que o custo de encontrar um pseudocaminho é $O(E)$, a complexidade total do algoritmo é $O(VE^2)$.

Perguntas?

`thiago.p.araujo@fgv.br`